Asynchronous Programming

Credit

- This slide is copied and modified from Fulvio Corno, Luigi De Russis @ Politecnico di Torino

Outline

- Callbacks
- Functional Programming
- Asynchronous Programming
- Database Access with SQLite
- Promises
- async/await

JavaScript: The Definitive Guide, 7th Edition
11.1 Asynchronous Programming with Callbacks

CALLBACKS

Callbacks

https://developer.mozilla.org/en-US/docs/Glossary/Callback_function

- A function passed into another function as an argument
  - Then invoked inside the outer function to complete some kind of routine or action
  - Synchronous
  - Asynchronous
- High-order function

```
1. function logQuote(quote) {
2.     console.log(quote);
3. }
4.
5. function createQuote(quote, callback) {
6.     const myQuote = `Like I always say,
   '${quote}'`;
7.     callback(myQuote);
8. }
9.
10.createQuote("WebApp I rocks!", logQuote);
```

Synchronous Callbacks

- Used in functional programming
  - e.g., providing the sort criteria for array sorting

```
1. let numbers = [4, 2, 5, 1, 3];
2. numbers.sort(function(a, b) {
3.    return a - b;
4. });
5. console.log(numbers);
```

```
1. let numbers = [4, 2, 5, 1, 3];
2. numbers.sort(( a, b ) => a – b);
3. console.log(numbers);
```

Synchronous Callbacks

- Example: filter according to a criteria
  - `filter()` creates a new array with all elements for which the callback returns true

```
1. const market = [
2.    { name: 'GOOG', var: -3.2 },
3.    { name: 'AMZN', var: 2.2 },
4.    { name: 'MSFT', var: -1.8 }
5. ];
6.
7. const bad = market.filter(stock => stock.var < 0);
8. // [ { name: 'GOOG', var: -3.2 }, { name: 'MSFT', var: -1.8 } ]
9.
10.const good = market.filter(stock => stock.var > 0);
11.// [ { name: 'AMZN', var: 2.2 } ]
```

JavaScript: The Definitive Guide, 7th Edition
Chapter 6. Array
Chapter 7.8 Functional Programming

FUNCTIONAL PROGRAMMING

Functional Programming: A Brief Overview

- A programming paradigm where the developer mostly construct and structure code using functions
  - Not JavaScript's main paradigm, but JavaScript is well suited
- More “declarative style” rather than “imperative style” (e.g., for loops)
- Can improve program readability

```
1. new_array =
2. array.filter( filter_function );
```

```
1. new_array = [] ;
2. for (const el of list)
3.    if ( filter_function(el) )
4.       new_array.push(el) ;
```

Notable Features of the Functional Paradigm

- Functions are first-class citizens
  - Functions can be used as if they were `variables` or `constants`, combined with other functions and generate new functions in the process, chained with other functions, etc.
- Higher-order functions
  - A function that operates on functions, taking one or more functions as arguments and typically returning a new function
- Function composition
  - Composing/creating functions to simplify and compress your functions by taking functions as an argument and return an output
- Call chaining
  - Returning a result of the same type of the argument, so that multiple functional operators may be applied consecutively

Functional Programming in JavaScript

- JavaScript supports the features of the paradigm “out of the box”
- Functional programming requires avoiding mutability
  - i.e., do not change objects in place!
  - e.g., if you need to perform a change in an array, return a new array

Iterating over Arrays

- Iterators: `for ... of, for (..;..;..)`
- Iterators: `forEach(f)`
  - Process each element with callback `f`
- Iterators: `every(f), some(f)`
  - Check whether all/some elements in the array satisfy the Boolean callback `f`
- Iterators that return a new array: `map(f), filter(f)`
  - Construct a new array
- `reduce`: callback function on all items to progressively compute a result
  - `reduce(callback(accumulator,currentValue[,index[,array]])[,initialValue])`

Iterables

- `Iterable` objects are a generalization of arrays
  - This concept allows us to make any object useable in a `for..of` loop
- Arrays are `iterable`
  - Many other objects are `iterable` as well. For instance, strings are also `iterable`
- Object represents a collection (`list`, `set`) of something, then `for..of` is a great syntax to loop over it

```
1. let range = {
2.   from: 1,
3.   to: 5
4. };
5.  // We want for(let num of range)
```

- What is a `Iterable`?
  - `Iterable` is an object that defines the Iterator property
  - Iterable is a data structure that makes its elements accessible through iteration
- Iterator is a pointer to the next element in the iteration
  - Iterators often used in combination with the `for...of` loop in JavaScript to iterate over elements

Symbol.iterator

- To make `range` object `iterable`, add a method named `Symbol.iterator`
- 1. When `for..of` starts, it calls that method once or errors if not found
  - The method must return an iterator – an object with the method next
- 3. When `for..of` wants the next value, it calls `next()` on that object.
- Result of `next()` must have the form `{done: Boolean, value: any}`
  - `done=true` means that the loop is finished, otherwise value is the `next` value

```
1. // 1. call to for..of initially calls this
2. range[Symbol.iterator] = function() {
3.  // ...it returns the iterator object:
4.  return {
5.   current: this.from,
6.   last: this.to,
7.   // 3. next() is called on each iteration
8.   next() {
9.    if (this.current <= this.last) {
10.     return {done: false, value: this.current++};
11.   } else {
12.     return { done: true }; // finished
13.   }
14.  }
15. };
16.};
```

Iterables and array-likes

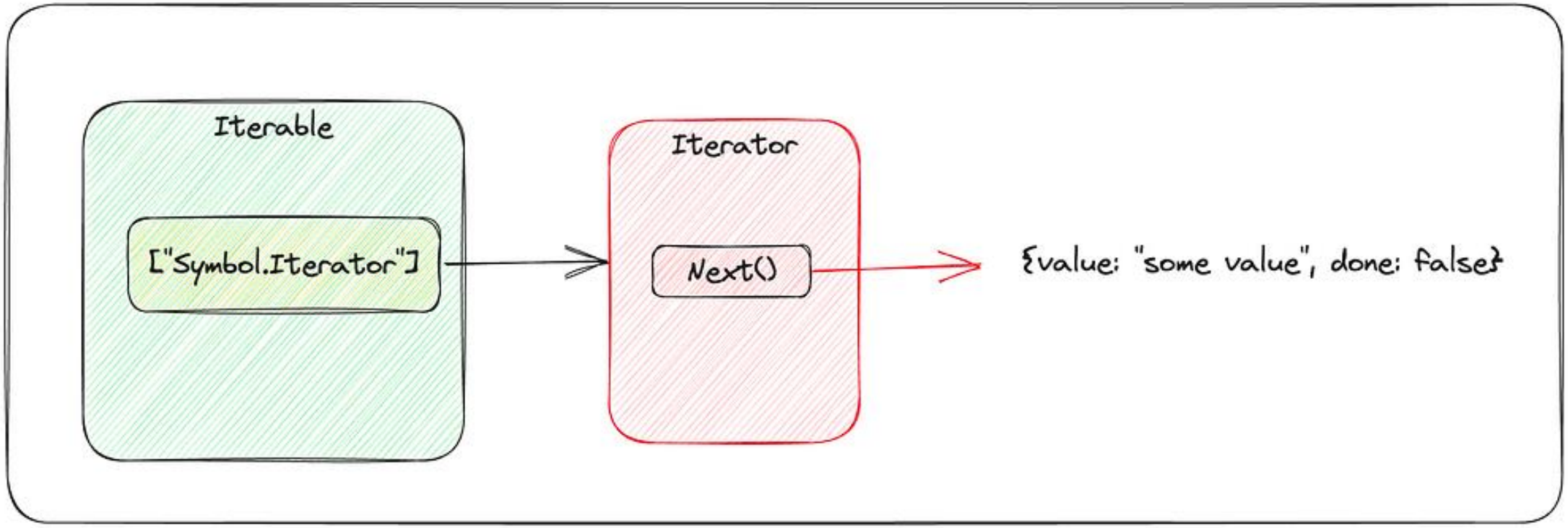
- `Iterables` are objects that implement the `Symbol.iterator` method
- Array-likes are objects that have indexes and length, so they look like arrays
- strings are both `iterable` and array-like
- the `range` in the example above is `iterable`, but not array-like

```
let arrayLike = {
  0: "Hello",
  1: "World",
  length: 2
}; // array-like, not iterable

// Error (no Symbol.iterator)
for (let item of arrayLike) {}
```

- `Array.from`
  - Universal method that takes an `iterable` or array-like value and makes a “real” array

```
let arrayLike = {
  0: "Hello",
  1: "World",
  length: 2
};

let arr = Array.from(arrayLike);
alert(arr.pop()); // World (method works)
```

Iterable and Iterator



.forEach()

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/forEach

- `forEach()` invokes your (synchronous) callback function once for each element of an `iterable`
  - `forEach(callback(currentElement, index, array), thisValue);`
- The callback may have **3** parameters

```
1. const letters = [..."Hello world"] ;
2. let uppercase = "" ;
3. letters.forEach(letter => {
4.    uppercase += letter.toUpperCase();
5. });
6. console.log(uppercase); // HELLO WORLD
7.
8. let numbers = [1,2,3,4,5];
9. numbers.forEach((num, index, arr) => {
10.   arr[index] = num * 2;
11.   // arr = [2, 4, 6, 8, 10]
12.});
13.console.log(numbers);
```

.forEach()

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/forEach

- The callback may have **3** parameters
  - `currentValue`: The current element being processed in the array
  - `index` (Optional): The index of `currentValue` in the array
  - `array` (Optional): The array `forEach()` was called upon
- Always returns `undefined` and is not chainable
- No way to stop or break a `forEach()` loop other than by throwing an exception
  - `forEach()` is 95% slower than a traditional `for` loop
- `forEach()` does not mutate the array on which it is called
  - however, its callback may do so

.every()

- `every()` tests whether all elements in the array pass the test implemented by the provided function
- `Callback`: Same 3 arguments as `forEach`
- It returns a **`Boolean`** value (`truthy/falsy`)
- It executes its callback once for each element present in the array until it finds the one where the callback returns a `falsy` value
  - If such an element is found, immediately returns `false`

```
1. let a = [1, 2, 3, 4, 5];
2. a.every(x => x < 10); // => true: all values are < 10
3. a.every(x => x % 2 === 0); // false: not all even values
```

.some()

- `some()` tests whether at least one element in the array passes the test implemented by the provided function
  - It returns a **`Boolean`** value
- It executes its callback once for each element present in the array until it finds the one where the callback returns a `truthy` value
  - if such an element is found, immediately returns true

```
1. let a = [1, 2, 3, 4, 5];
2. a.some(x => x%2 === 0); // => true; a has some even numbers
3. a.some(isNaN);
```

.map()

- **map()** passes each element of the array on which it is invoked to the function you specify
  - Callback should return a value
- **map()** always returns a new array containing the values returned by the callback

```
1. const a = [1, 2, 3];
2. const b = a.map(x => x*x);
3. console.log(b); // [1, 4, 9]
4.
5. const letters = ["Hello world"];
6. const uppercase = letters.map(
7.    letter => letter.toUpperCase()
8. );
9. console.log(uppercase.join(''));
```

.filter()

- **`filter()`** creates a new array with all elements that pass the test implemented by the provided function
  - Callback is a function that returns either true or false
  - If no element passes the test, an empty array is returned

```
1. const a = [5, 4, 3, 2, 1];
2. a.filter(x => x < 3);
3. // generates [2, 1], values less than 3
4. a.filter( (element, index) => index % 2 == 0 );
5. // [5, 3, 1]
```

.reduce()

- `reduce()` **combines** elements of an array, using the specified function, to produce a **single** value
  - A common operation in functional programming and goes by the names "inject" and "fold"

```
1. reduce(
2.    callback(
3.       accumulator,
4.       currentValue[, index[, array]]
5.    )
6.    [, initialValue]
7. }
```

- `reduce()` takes **2** arguments:
  - "reducer function" (callback) that performs reduction/combination operation
    - Combine or reduce 2 values into 1
  - `initialValue` (optional) to pass to the function
    - If not specified, it uses the **first** element of the array as initial value and
    - Iteration starts from the next element

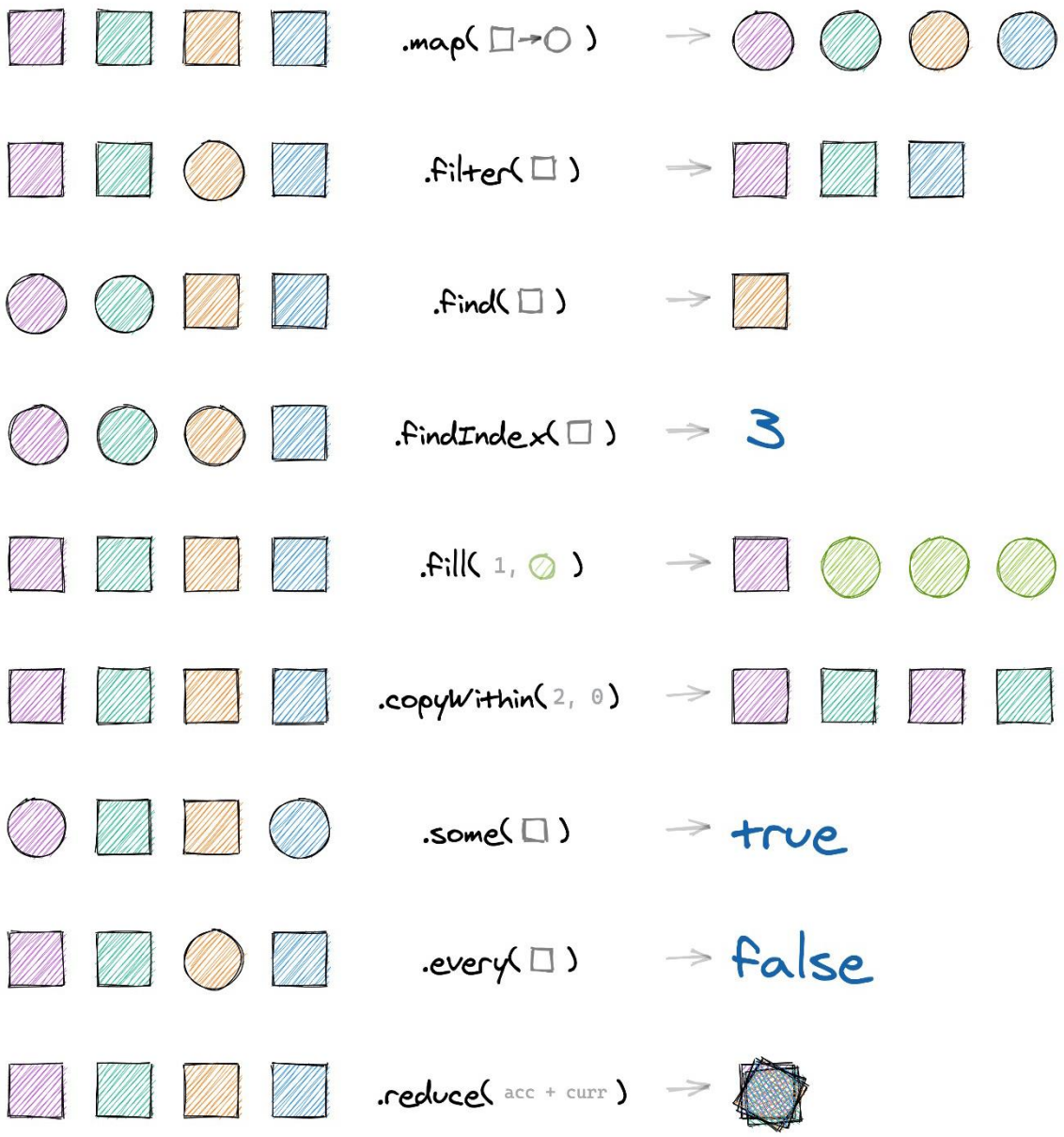
.reduce()

- `Callbacks` used with `reduce()` are different than the ones used with `forEach()` and `map()`
  - First argument is the accumulated result of the reduction so far
  - On the first call to this function, its first argument is the initial value
  - On subsequent calls, it is the value returned by the previous invocation of the reducer function

```
1. const a = [5, 4, 3, 2, 1];
2. a.reduce( (accumulator, currentValue) =>
3.    accumulator + currentValue, 0
4. ); // 15; the sum of the values
5. a.reduce((acc, val) => acc*val,  1);
6.    // 120; the product of the values
7. a.reduce( (acc, val) =>
8.    (acc > val) ? acc : val
9. );
10.   // 5; the largest of the values
```

Array methods cheatsheet

JS tips
@sulco



JavaScript: The Definitive Guide, 7th Edition
Chapter 11. Asynchronous JavaScript

Mozilla Developer Network

- Learn web development JavaScript » Dynamic client-side scripting » Asynchronous JavaScript
- Web technology for developers » JavaScript » Concurrency model and the event loop
- Web technology for developers » JavaScript » JavaScript Guide » Using Promises

ASYNCHRONOUS PROGRAMMING

Asynchronicity

- JavaScript is **single-threaded** and inherently **synchronous**
  - i.e., code cannot create threads and run in parallel in the JS engine
- Callbacks are the most fundamental way for writing **asynchronous** JS code
- How can they work asynchronously?
  - e.g., how can **`setTimeout()`** or other async callbacks work?
- Thanks to the Execution Environment
  - e.g., browsers and Node.js
- and the Event Loop

```
1. const deleteAfterTimeout = (task) => {
2.    // do something
3. }
4. // runs after 2 seconds
5. setTimeout(deleteAfterTimeout, 2000, task)
```

Non-Blocking Code!

- Asynchronous techniques are useful, particularly for web development
  - e.g., when a web app runs executes an intensive chunk of code without returning control to the browser, the browser can appear to be frozen
  - This is called blocking, and it should be the exception!
    - Browser is blocked from continuing to handle user input and perform other tasks until the web app returns control of the processor
- This may happen outside browsers, as well
  - e.g., reading a long file from the disk/network, accessing a database and returning data, accessing a video stream from a webcam, etc.
- Most of the JS execution environments are, therefore, deeply asynchronous
  - with non-blocking primitives
  - JavaScript programs are event-driven, typically

Asynchronous Callbacks

- The most fundamental way for writing asynchronous JS code
  - Great for “simple” things!
- Handling user actions
  - e.g., button click
- Handling I/O operations
  - e.g., fetch a document
- Handling time intervals
  - e.g., timers
- Interfacing with databases

```
1. const readline = require('readline');
2. const rl = readline.createInterface({
3.    input: process.stdin,
4.    output: process.stdout
5. });
6. rl.question('How old are you? ', (answer) => {
7.    let description = answer;
8.    rl.close();
9. });
10. --------------------
11.let input;
12.rl.on('line', (line) => {
13.   input = line;
14.   rl.close();
15.}
16.console.log(input);
17.undefined
18.> hi
```

Timers

- To delay the execution of a function:
  - `setTimeout()` runs the callback function after a given period of time
  - `setInterval()` runs the callback function periodically

```
1. const onesec = setTimeout(()=> {
2.    console.log('hey') ; // after 1s
3. }, 1000) ;
4. console.log('hi') ;
5.
6. const myFunction = (firstParam, secondParam) => {
7.    // do something
8. }
9. // runs after 2 seconds
10.setTimeout(myFunction, 2000, firstParam, secondParam) ;
```

Timers

- `clearInterval()` for stopping the periodical invocation of `setInterval`

```
1. const id = setInterval(() => {}, 2000) ;
2. // «id» is a handle that refers to the timer
3.
4. clearInterval(id) ;
```

Handling Errors in Callbacks

- No “official” ways, only best practices!
- First parameter of callback function is for storing any error
  - Second one is for the result of the operation
  - this is the strategy adopted by Node.js

```
1. fs.readFile('/file.json', (err, data) => {
2.    if (err !== null) {
3.       console.log(err);
4.       return;
5.    }
6.    //no errors, process data
7.    console.log(data);
8. });
```

PROMISES

JavaScript: The Definitive Guide, 7th Edition
Chapter 11. Asynchronous JavaScript

Mozilla Developer Network

- Learn web development JavaScript » Dynamic client-side scripting » Asynchronous JavaScript
- Web technology for developers » JavaScript » Concurrency model and the event loop
- Web technology for developers » JavaScript » JavaScript Guide » Using Promises

Beware: *Callback Hell*!

- Must keep passing new functions to perform multiple asynchronous actions in a row using callbacks
  - Every callback adds a level of nesting
  - When you have lots of callbacks, code starts to be complicated very quickly
- I am requesting some data from the server by calling `getData()`
  - Inside I am requesting for some more data by calling `getMoreData()` passing previously received data as an argument

```
1. getData(function(x){
2.     console.log(x);
3.     getMoreData(x, function(y){
4.         console.log(y);
5.         getSomeMoreData(y, function(z){
6.             console.log(z);
7.         });
8.     });
9. });
```

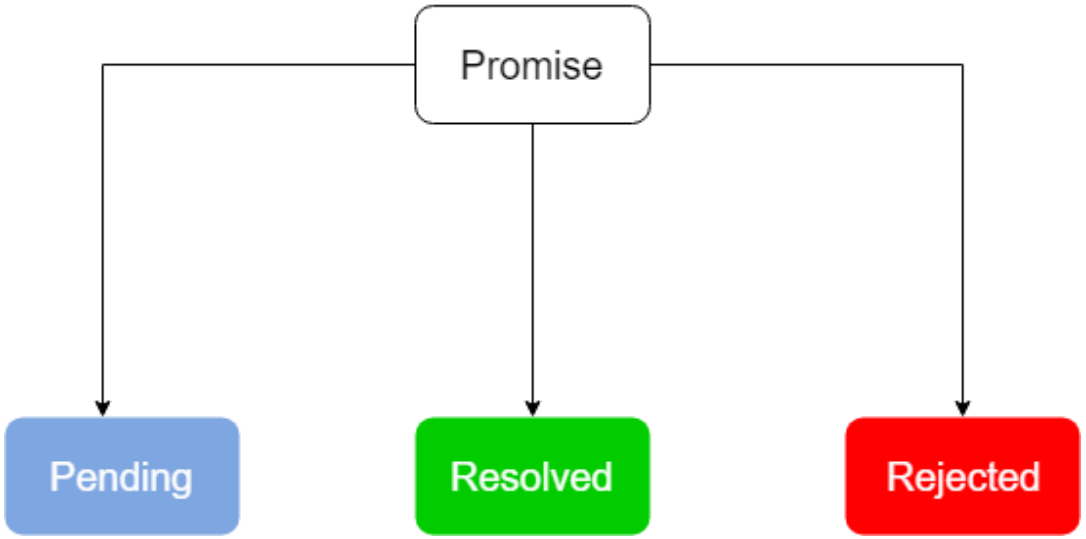
```
1. getData()      // Promise
2.   .then((x) => {
3.     console.log(x);
4.     return getMoreData(x);
5.   })
6.   .then((y) => {
7.     console.log(y);
8.     return getSomeMoreData(y);
9.   })
10.  .then((z) => {
11.    console.log(z);
12.  });
```

Promises

- A core language feature to “simplify” asynchronous programming
  - Promise is an object that holds the future value of an async operation
    - If we are requesting some data from a server, Promise promises us to get that data which we can use in future
- It is an object representing the eventual completion (or failure) of an asynchronous operation
  - i.e., an asynchronous function returns a promise to supply the value at some point in the future, instead of returning immediately a final value
- Promises standardize a way to handle errors and provide a way for errors to propagate correctly through a chain of promises

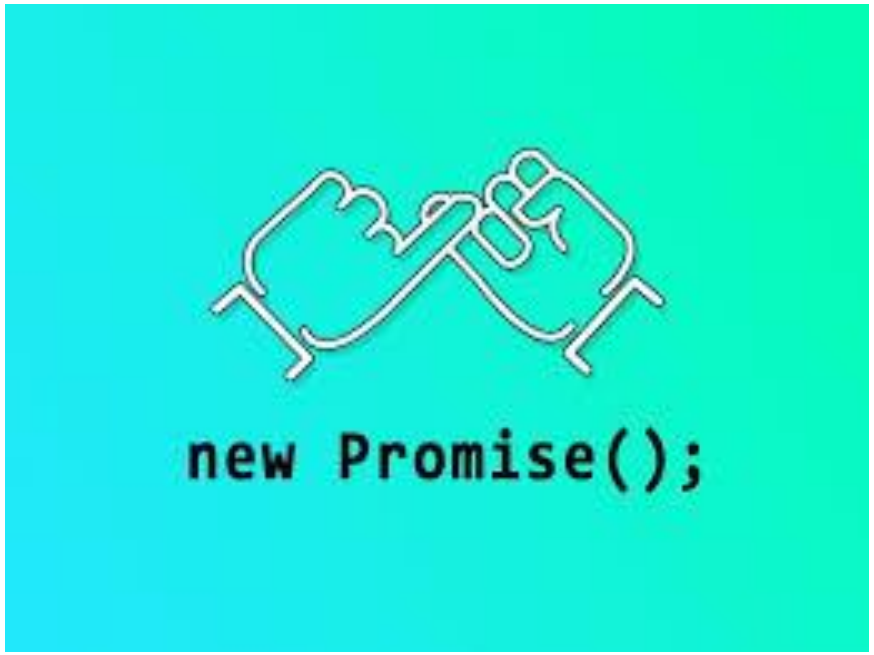
States of a Promise

- Unresolved or Pending
  - Result is not ready
  - Waiting for something to finish (for example, an async operation)
- Resolved or Fulfilled
  - A Promise is resolved if the result is available
  - Something finished and all went well.
- Rejected
  - A Promise is rejected if an error happened



Promises

- Promises can be created or consumed
  - many Web APIs expose Promises to be consumed!
- When consumed:
  - Promise starts in a pending state
    - Caller function continues the execution, while it waits for the Promise to do its own processing, and give the caller function some “responses”
  - Caller function waits for it to either return the promise in a fulfilled state
    or in a rejected state



Creating a Promise

- Created using the **new** keyword with the Promise constructor
  - It takes a single argument, a callback, also known as **executor** function which takes two callbacks, resolve and reject
  - The executor function is immediately executed when a promise is created
- Takes two functions as parameters:
  - **resolve**, called when the asynchronous task completes successfully and returns the results of the task as a value
  - **reject**, called when the task fails and returns the reason for failure (an error object, typically)

```
1. const promise =
2.    new Promise((resolve, reject) => {
3.    const randomNumber = Math.random();
4.    setTimeout(() => {
5.       if(randomNumber < .6) {
6.          resolve('All things went well!');
7.       } else {
8.          reject('Something went wrong');
9.       }
10.   }, 2000);
11.});
```

Promise.resolve( )

- `Promise.resolve()` static method "resolves" a given value to Promise
  - If the value is a promise, that promise is returned
  - if the value is thenable it will call `then()` method with two callbacks
    - Otherwise the returned promise will be fulfilled with the value

```
1. const p = Promise.resolve([1, 2, 3]);
2. p.then((v) => {
3.    console.log(v[0]); // 1
4. });
```

Example

- Promise is resolved or rejected after 2 seconds
  - Resolved if the `randomNumber` is less than .6 and rejected otherwise
- When Promise is created, it will be in the pending state, its value will be undefined
- After the 2 seconds, Promise is either resolved or rejected state
  - Its value will be the value passed to the resolve or reject function
- Executor calls only one resolve or one reject
  - All further calls of resolve and reject are ignored

```
1. const promise =
2.    new Promise((resolve, reject) => {
3.    const randomNumber = Math.random();
4.    setTimeout(() => {
5.       if(randomNumber < .6) {
6.          resolve('All things went well!');
7.       } else {
8.          reject('Something went wrong');
9.       }
10.   }, 2000);
11.});
```

Consuming a Promise

- Promise serves as a link between the executor and the consuming functions
  - Consuming functions can be registered using **`then()`** and **`catch()`** methods

`.then(): promise.then(successCB, failCB)`

  - Promise is resolved, the **`successCB`** is called with the value passed to **`resolve()`**
  - Promise is rejected, the **`failCB`** is called with the value passed to **`reject()`**

```
1. const promise =
2.   new Promise((resolve, reject) => {
3.   const randomNumber = Math.random();
4.
5.   if(randomNumber < .7) {
6.     resolve('All things went well!');
7.   } else {
8.     reject(new Error('Something went wrong'));
9.   }
10.});
11.
12.promise.then((data) => {
13.  console.log(data);
14.   // prints 'All things went well!'
15.  }, // sucessCB
16.  (error) => {
17.  console.log(error); // prints Error object
18.  } // failCB
19.);
```

Consuming a Promise

`.catch(): promise.catch(failCB)`

- Use **`catch()`** for handling errors
  - More readable than handling errors inside the **`failCB`** of the **`then()`** callback

```
1. const promise = new Promise((resolve, reject) => {
2.   reject(new Error('Something went wrong'));
3. });
4. promise
5.   .then((data) => {
6.     console.log(data);
7.    })
8.   .catch((error) => {
9.     console.log(error); // prints Error object
10.  });
```

- You can omit **`catch()`**, if you are interested in the result, only
- If we're interested only in errors, we can use **`null`** as the first argument
  - `.then(null, errorFunction)`
- or can use
  - `.catch(errorFunction)`

Consuming a Promise

- **`p.then(onFulfilled[, onRejected]);`**
  - Callbacks are executed asynchronously (inserted in the event loop) when the promise is either fulfilled (success) or rejected (optional)
- **`p.catch(onRejected);`**
  - Callback is executed asynchronously (inserted in event loop) when promise is rejected
  - Short for **`p.then(null, failureCallback)`**
- **`p.finally(onFinally);`**
  - Callback is executed in any case, when the promise is either fulfilled or rejected
  - Useful to avoid code duplication in then and catch handlers
- All these methods return Promises, too! ⇒ They can be chained

Promise: Create & Consume

```
const prom = new Promise(
   (resolve, reject) => {
      ...
      resolve(x);
      ...
      reject(y);
      ...
  }
)
```

```
prom
   .then((x) => {
      ...use x...
   } )
   .catch( (y) => {
      ...use y...
   } ) ;
```

Promise Chaining

- Chaining resolves promises in a sequence
- Resolving `promise1` call `then()` which returns `promise2`
- Next `then()` is called which returns `promise3`
- `Promise3` is rejected
  - `catch()` is called which handles the `promise3` rejection
  - Generally only one `catch()` is enough for handling rejection

```
1. const promise1 = new Promise((resolve, reject) => {
2.   resolve('Promise1 resolved');
3. });
4. const promise2 = new Promise((resolve, reject) => {
5.   resolve('Promise2 resolved');
6. });
7. const promise3 = new Promise((resolve, reject) => {
8.   reject('Promise3 rejected');
9. });
10.promise1
11.   .then((data) => {
12.     console.log(data);  // Promise1 resolved
13.     return promise2;
14.   })
15.   .then((data) => {
16.     console.log(data);  // Promise2 resolved
17.     return promise3;
18.   })
19.   .then((data) => {
20.     console.log(data);
21.   })
22.   .catch((error) => {
23.     console.log(error);  // Promise3 rejected
24.   });
```

Chaining Promises

- They provide a natural way to express a sequence of asynchronous operations as a linear chain of **`then()`** invocations
  - **Without having to nest** each operation within the callback of the previous one
    - The "callback hell" seen before
  - If error handling code is same for all steps, you can attach **`catch()`** to **end of chain**
- Important: always return results, otherwise callbacks won't get the result of a previous promise

Promise.all()

- It takes an array of promises as input and returns a new promise
  - Fulfills when all of the promises inside the input array have fulfilled or
  - Rejects as soon as one of the promises in the array rejects
- Data inside `then()` is an array which contains promise values in the same order as defined in promise array passed to `Promise.all()`
  - `Promise1 resolved Promise2 resolved`

```
1. const promise1 = new Promise((resolve, reject) => {
2.     setTimeout(() => {
3.         resolve('Promise1 resolved');
4.     }, 2000);
5. });
6. const promise2 = new Promise((resolve, reject) => {
7.     setTimeout(() => {
8.         resolve('Promise2 resolved');
9.     }, 1500);
10.});
11.Promise.all([promise1, promise2])
12.    .then((data) => console.log(data[0], data[1]))
13.    .catch((error) => console.log(error));
```

Promise.all()

- Promise is rejected with the reason of rejection from the first rejected promise in the input array
- As soon as the second promise is rejected after 1.5 seconds, the returned promise from `Promise.all()` is rejected without waiting for the first promise
- So `Promise.all()` waits for all promises to succeed and fails if any of the promises in the array fails

```
1. const promise1 = new Promise((resolve, reject) => {
2.     setTimeout(() => {
3.         resolve('Promise1 resolved');
4.     }, 2000);
5. });
6. const promise2 = new Promise((resolve, reject) => {
7.     setTimeout(() => {
8.         reject('Promise2 rejected');
9.     }, 1500);
10.});
11.Promise.all([promise1, promise2])
12.    .then((data) => console.log(data[0], data[1]))
13.    .catch((error) => console.log(error));
```

Promise.race()

- It takes an array of promises as input and return new promise that
  - Fulfills as soon as one of the promises in the input array fulfills or
  - Rejects as soon as one of the promises in the input array rejects
- As soon as the first promise is resolved after 1 second
  - Returned promise from `Promise.race()` is resolved without waiting for the second promise to be resolved or rejected

```
1. const promise1 = new Promise((resolve, reject) => {
2.     setTimeout(() => {
3.     resolve('Promise1 resolved');
4.     }, 1000);
5. });
6. const promise2 = new Promise((resolve, reject) => {
7.     setTimeout(() => {
8.         reject('Promise2 rejected');
9.     }, 1500);
10.});
11.Promise.race([promise1, promise2])
12.  .then((data) => console.log(data))  // resolved
13.  .catch((error) => console.log(error));
```

Example: Chaining

- Useful, for instance, with I/O API such as **`fetch()`**, which returns a Promise

```
1. const status = (response) => {
2.    if (response.status >= 200 && response.status < 300) {
3.       return Promise.resolve(response)
4.       // static method to return a fulfilled Promise
5.    }
6.    return Promise.reject(new Error(response.statusText))
7. }
8. const json = (response) => response.json()

9. fetch('/todos.json’)
10.   .then(status)
11.   .then(json)
12.   .then((data) => { console.log('Request succeeded with JSON response', data) })
13.   .catch((error) => { console.log('Request failed', error) })
```

ASYNC/AWAIT

JavaScript: The Definitive Guide, 7th Edition
Chapter 11. Asynchronous JavaScript

Mozilla Developer Network

- Learn web development JavaScript » Dynamic client-side scripting » Asynchronous JavaScript
- Web technology for developers » JavaScript » Concurrency model and the event loop
- Web technology for developers » JavaScript » JavaScript Guide » Using Promises

How It Works

- `async/await` syntax is a special syntax created to help you work with `promise`
  - Need to chain calls to function or variable that returns a `Promise` using `then`/`catch` methods
- When you have many promises, you will also need lots of then method chains

- `fetch()` function returns a Promise, which we handle using `then()` method
  - First `then()`, we accept the response from the request and convert it into an object
  - Second `then()`, we receive the returned json then log that data to the console.
  - `Catch()` method handles any error that might happen

```
1. fetch('https://jsonplaceholder.typicode.com/todos/1')
2.   .then(response => response.json())
3.   .then(json => console.log(json))
4.   .catch(error => console.log(error));
```

Fetch API

- Simple interface for fetching resources
- It allows us to make network request and handle responses easier than our old `XMLHttpRequest(XHR)`
  - Fetch API uses Promises, which provides a way to avoid callbacks hell and boilerplate heavy code that XHR provides

- Promise, returned by fetch, resolves with an object of the built-in Response class

```
1. let promise = fetch(url, [options])
```

Await

```
1. fetch('https://jsonplaceholder.typicode.com/todos/1')
2.   .then(response => response.json())
3.   .then(json => console.log(json))
4.   .catch(error => console.log(error));
```

- `await` makes JavaScript **wait** until the Promise object is resolved or rejected
  - Instead of having to use the callback pattern inside `then()`, you can assign the fulfilled promise into a variable

```
1. const response = await
   fetch('https://jsonplaceholder.typicode.com/todos/1');
2. const json = await response.json();
3. console.log(json)
```

- `await` keyword is placed before call to a function or variable that returns promise
  - It makes JavaScript wait for promise object to settle before running code in next line

- SyntaxError
  - `await is only valid in async functions and the top level bodies of modules`

Simplifying Writing With **`async/await`**

- ECMAScript 2017 (ES8) introduces two new keywords, **`async`** and **`await`**
  - Write promise-based asynchronous code that looks like synchronous code
- Prepend **`async`** keyword to any function means that it will return **`Promise`**
- Prepend **`await`** when calling an **`async`** function (or a function returning a **`Promise`**) makes the calling code stop until promise is resolved or rejected

```
1. const sampleFunction = async () => {
2.    return 'test'
3. }
4. sampleFunction().then(console.log)
5. // This will log 'test'
```

Async

- You add `async` keyword before function name to create asynchronous function
- We created an asynchronous function called `runProcess()` and put the code that uses the `await` keyword inside it
  - We can then run the asynchronous function by calling it

```
1. async function runProcess() {
2.    const response = await
3.    fetch('https://jsonplaceholder.typicode.com/todos/1');
4.    const json = await response.json();
5.    console.log(json)
6. }
7.
8. runProcess();
```

How to Handle Errors in `Async/Await`

- You can use the `try/catch` block to catch any rejection from your promise
- `try/catch` block, put inside the `runProcess()` function, will handle the rejection that comes from the promise

```
1. async function runProcess() {
2.    try {
3.       const response = await
4.       fetch('https://jsonplaceholder.typicode.com/todos/1');
5.       const json = await response.json();
6.       console.log(json);
7.    } catch (error) {
8.       console.log(error);
9.    }
10.}
11.runProcess();
```

PROMISE

```
fetch('https://jsonplaceholder.typicode.com/todos/1')
  .then(response => response.json())
  .then(json => console.log(json))
  .catch(error => console.log(error));
```

ASYNC/AWAIT

```
async function runProcess() {
  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/todos/1');
    const json = await response.json();
    console.log(json);
  } catch (error) {
    console.log(error);
  }
}

runProcess();
```

Examples: Before and After

```
1. const makeRequest = () => {
2.    return getAPIData()
3.     .then(data => {
4.       console.log(data);
5.       return "done";
6.       }
7.    );
8. }
9.
10.let res = makeRequest();
```

- Arrow function

```
1. const makeRequest = async () => {
2.    console.log(await getAPIData());
3.    return "done";
4. };
5.
6. let res = makeRequest();
```

Examples: Before and After

```
1. function getData() {
2.    return getIssue()
3.       .then(issue => getOwner(issue.ownerId))
4.       .then(owner => sendEmail(owner.email, 'Some text'));
5. }
6. // assuming that all the 3 functions above return a Promise
```

```
1. async function getData = {
2.    const issue = await getIssue();
3.    const owner = await getOwner(issue.ownerId);
4.    await sendEmail(owner.email, 'Some text');
5. }
```

Chaining with async/await

- Simpler to read, easier to debug
  - Debugger would not stop on asynchronous code

```
1. const getFirstUserData = async () => {
2.    const response = await fetch('/users.json'); // get users list
3.    const users = await response.json(); // parse JSON
4.    const user = users[0]; // pick first user
5.    const userResponse = await fetch(`/users/${user.name}`); // get user data
6.    const userData = await user.json(); // parse JSON
7.    return userData;
8. }
9. getFirstUserData();
```

License

- These slides are distributed under a Creative Commons license “**Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)**”
- **You are free to:**
  - **Share** — copy and redistribute the material in any medium or format
  - **Adapt** — remix, transform, and build upon the material
  - The licensor cannot revoke these freedoms as long as you follow the license terms.
- **Under the following terms:**
  - **Attribution** — You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
  - **NonCommercial** — You may not use the material for commercial purposes.
  - **ShareAlike** — If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.
  - **No additional restrictions** — You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.
- https://creativecommons.org/licenses/by-nc-sa/4.0/